Q: Who has done lab 1?
Any questions/comments?

# Distributed Systems & Networking (part 1, inc. UDP)

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

**Notices to share with students at the start of an EchoPoll session**

The following notices are taken from the 'Code of Practice for the Safe Use of EchoPoll v1.1'

While text responses may sometimes appear to be anonymous to the audience, they are always traceable

Please be advised that names and e-mail addresses of respondents may be displayed on screen

# Contents

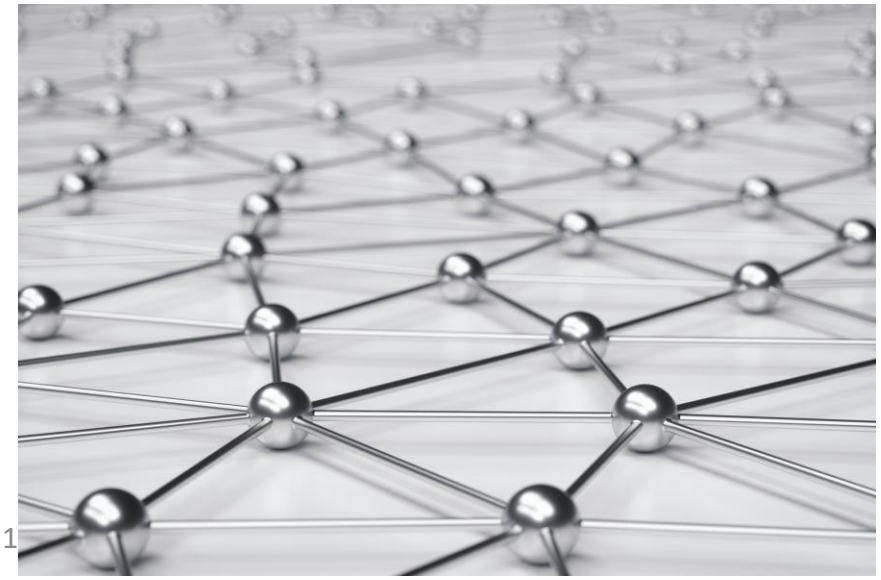
- Distributed Systems (cont.)
- The Internet: IP, TCP and UDP (revision)
- Java UDP Programming
- Request-Response Protocols

| 5 | **Application** |
|---|---|
| 4 | **Transport (TCP, UDP)** |
| 3 | **Internet (IP)** |
| 2 | Network interface / Link |
| 1 | Physical |

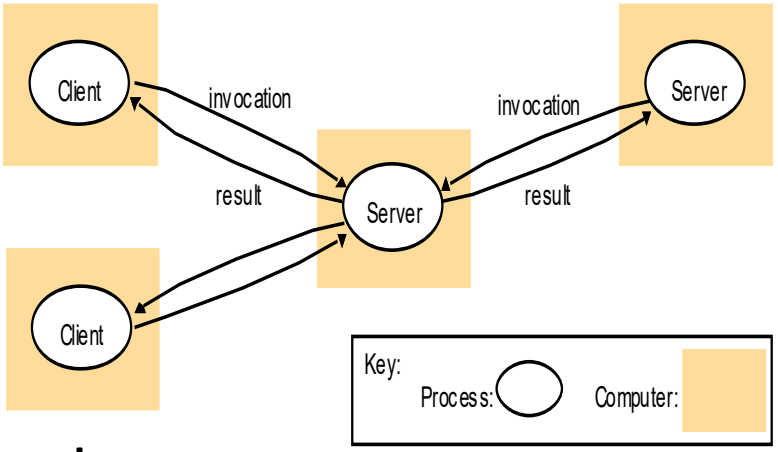
# Distributed Systems (cont.)

# Definition (recap)

- *"We define a distributed system as one in which hardware or software **components** located at **networked computers** communicate and coordinate their actions only by passing **messages**."*
  - Distributed Systems: Concepts and Design (Edn. 5), Coulouris, Dollimore, Kindberg and Blair



# Services

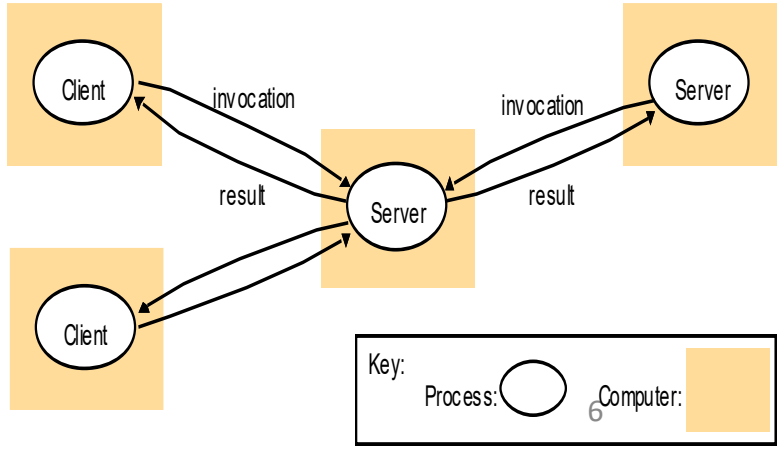



- A **service** is "a distinct part of a computer system that manages a collection of related **resources** and presents their functionality to users and applications."
  - It runs on the specific machine which physically hosts the managed resources
  - It provides a specific and controlled **interface** – usually over a network – to the resources it manages
- Many distributed systems consist only of **clients** making requests to **servers** to access or manipulate the resources managed by that **service**
  - E.g. Web browser(s) (client) making requests to web server(s)

# Talking about distributed system **architecture**…

(see later lecture on models)

- What are the **entities** that are communicating in the distributed system?
  → Often Operating System processes
- How do they **communicate**, or, more specifically, what *communication paradigm* is used?
  → Often direct, request-response communication
- What (potentially changing) **roles and responsibilities** do they have in the overall architecture?
  → Often client and server
- How are they mapped on to the physical distributed infrastructure (i.e. what is their ***placement***)?
  → e.g. on user desktop and cloud server



# Distributed System Challenges: Heterogeneity

(Common challenges for distributed systems, e.g. when defining protocols):

- All the parts of a distributed system have to work together, even if some are made differently
  - e.g. different network technologies, CPUs, architectures, operating systems, programming languages, developers
- Typically this will mean using common protocols, interfaces and encodings
  - perhaps supported by common libraries, tools and services.
- E.g. Java Data Input/Output Streams
  - See also External Data Representation (next lecture)

# Failure Handling
## (see also later lectures on Failure & Reliability)

- Distributed systems can fail in complicated ways,
  - i.e. different parts can fail at different times
- Coping with failure typically requires
  - some level of redundancy, i.e. spare or replicated resources
  - specific means detecting and responding to partial failures
- E.g. simple TCP or UDP server
  - Deploy multiple servers?
  - Retrying failed connections?
    - (UDP: resending lost datagrams)

# Concurrency
(see also Operating Systems and Concurrency module)

- The different processes in a distributed system normally run concurrently
- This may require additional checks and communication to ensure global consistency
  - these can limit performance or scalability (c.f. locking)
- Or use alternative approaches with weaker consistency
  - e.g. loosely coupled or optimistic operations
- E.g. the example TCP server (next lecture) is multi-threaded so that it can serve multiple **independent** clients concurrently

# Distributed System Challenges

Common challenges for distributed systems (e.g. when defining protocols):

- **Heterogeneity** – coping with system component variability
- **Failure Handling** – coping with partial failure
- **Concurrency** – correctness and performance with concurrency

And (to look at in a later lecture):

- Openness – to extension or reproduction
- Security – protection from threats
- Scalability – good performance as systems grow
- Transparency – selective abstraction
- Quality of Service - performance & other non-functional characteristics

| 5 | Application |
|---|---|
| 4 | **Transport (TCP, UDP)** |
| 3 | **Internet (IP)** |
| 2 | Network interface / Link |
| 1 | Physical |

# The Internet: IP, TCP and UDP

Should just be revision :-)

# Network Protocol Stacks

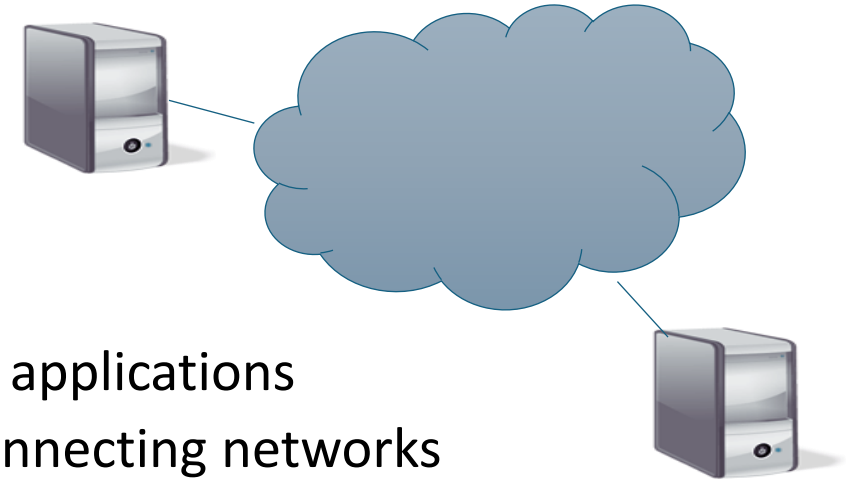
- Structed (layered) arrangement of protocols, where each layer:
  - is a set of protocols (or one protocol) and
  - makes use of the services offered by the protocols of the layer below.

| 5 | Application |
|---|---|
| 4 | Transport (TCP, UDP) |
| 3 | Internet (IP) |
| 2 | Network interface / Link |
| 1 | Physical |

# Network Protocol

- Set of rules about how to communication, e.g.
  - What a service user can ask for
  - What information to send
  - When to send it
  - How to encode the information
  - What to do if something goes wrong
- => If everyone follow the same rules then effective communication is achieved

# The Internet



- The Internet is:
  - (1) The illusion of a single network provided to users and applications
  - (2) The underlying physical structure with routers interconnecting networks
- Based on specific layer 3 and layer 4 protocols plus supporting layer 5 protocols (DNS, routing)
  - Open standards managed by the Internet Engineering Task Force (IETF)
- It provides a unifying point for almost any network application
  - It doesn't care what technologies/protocols it is built on (layers 1 and 2)
  - It doesn't care what applications it is used for (layer 5)

| 5 | **Application** |
|---|---|
| 4 | **Transport (TCP, UDP)** |
| 3 | **Internet (IP)** |
| 2 | Network interface / Link |
| 1 | Physical |

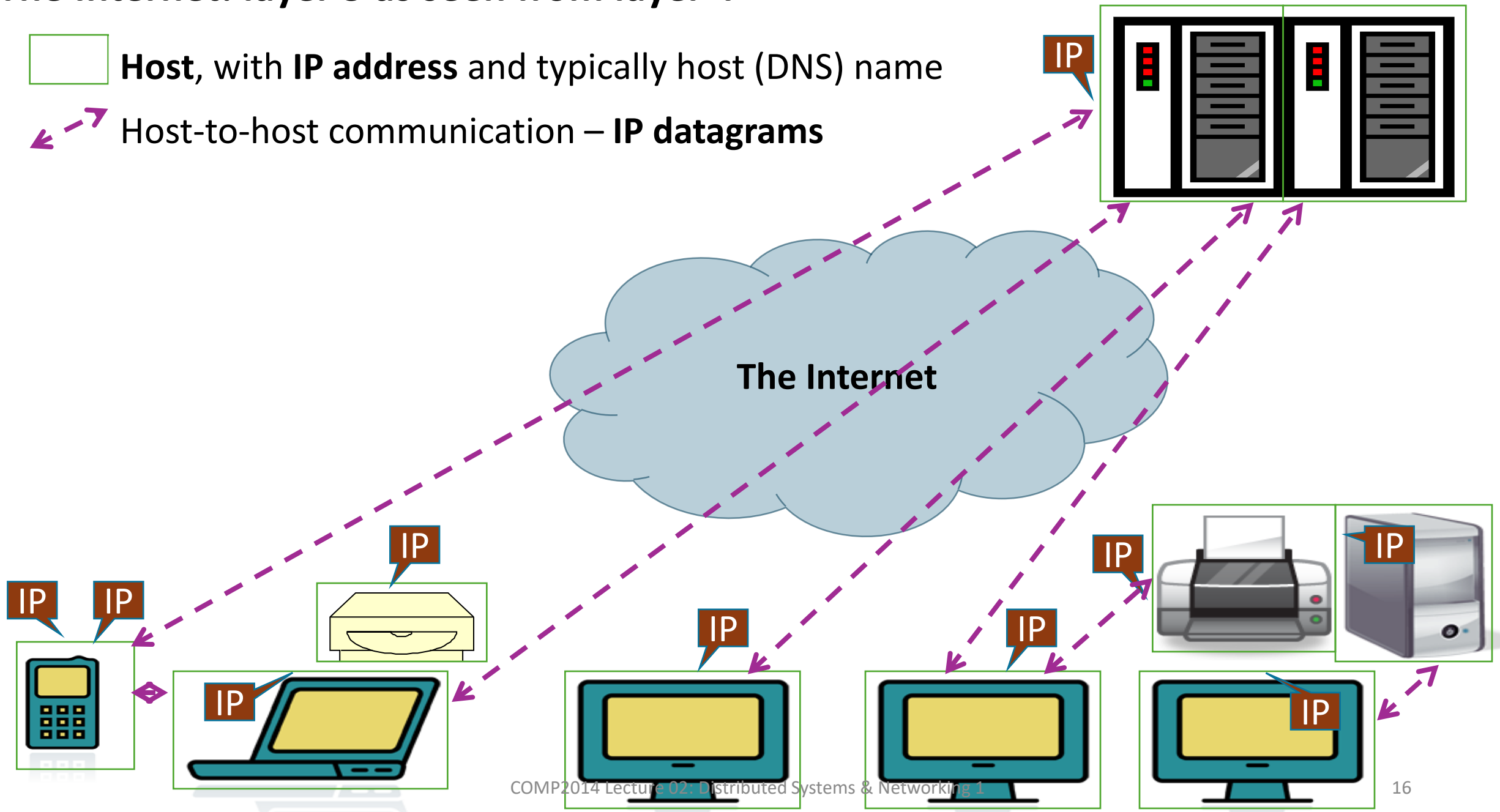
# Typical network protocol stack

| Layer | Name | Common Protocol(s) | Scope |
|---|---|---|---|
| 5 | Application | HTTP, **DNS** (**DHCP**) ((**routing**)) | Application-specific (including make your own…) |
| **4** | **Transport** | **TCP, UDP** | Generic packets and streams |
| **3** | **Internet** | **IP (ICMP, ARP)** | Global communication |
| 2 | Network Interface (Link) | 802.11 – Ethernet, WiFi | Single hop (link) communication |
| 1 | Physical | Ethernet PHY, ADSL, … | Signals & physics |

# The Internet: layer 3 as seen from layer 4

**Host**, with **IP address** and typically host (DNS) name

Host-to-host communication – **IP datagrams**



# The Internet model

- Every networked machine is a **node** or **host**
- Every host has one or more **Internet** (**IP**) **addresses**
  - (One for each network interface – see next lecture)
  - E.g. “128.243.22.73”
    - (a new version of IP is being rolled out, IPv6, which uses longer addresses, e.g. “2001:db8:85a3:8d3:1319:8a2e:370:7348”)
- Any machine can send a **packet** of data to any other machine (aka a **datagram**)
  - Packet size is limited to 64kB, often less
- Packets may be lost, delayed, corrupted or duplicated, although there is some checking for corrupted data
  - This is termed a **best effort** service

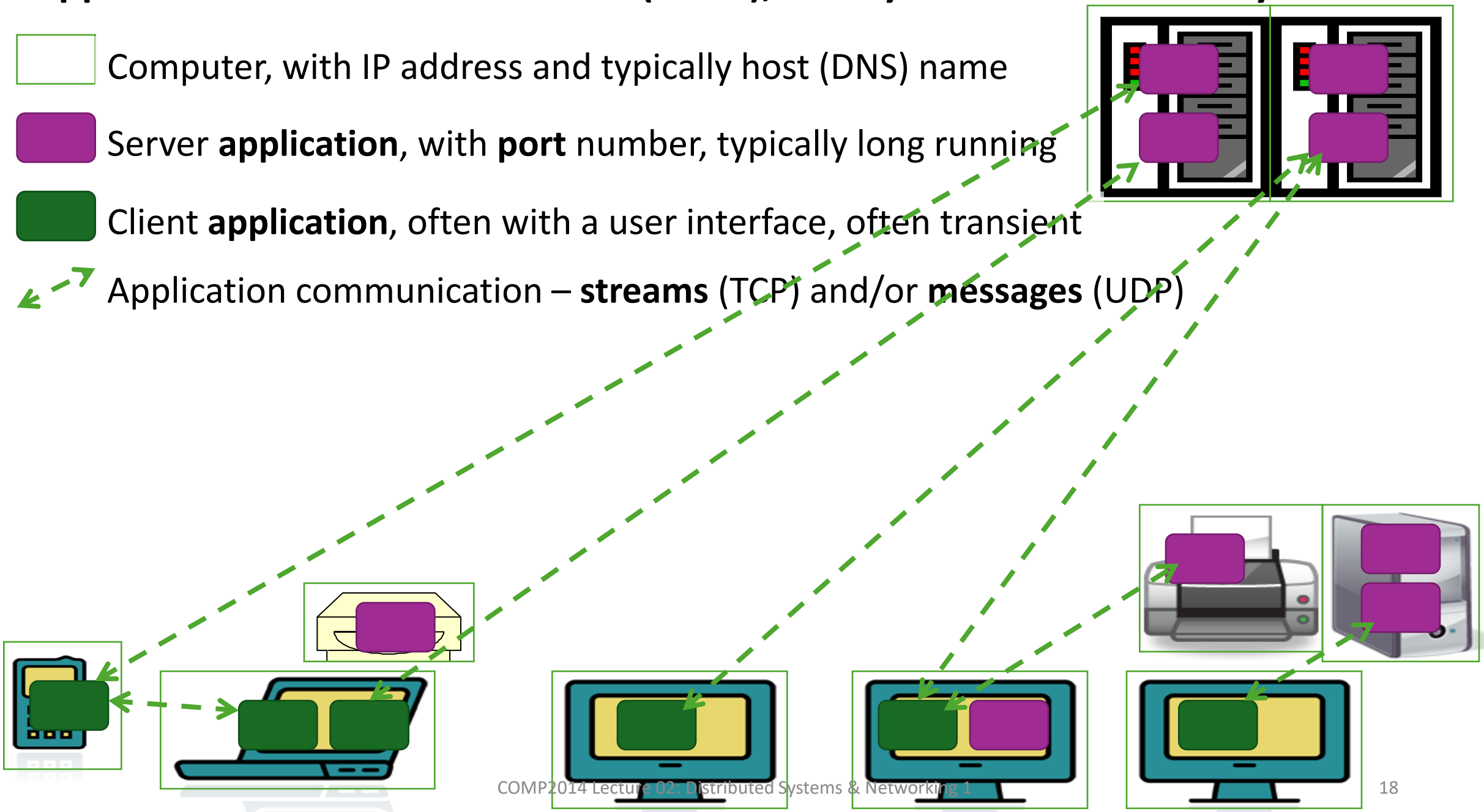
# An Applications View of the Internet (Part I), i.e. layer 4 as seen from layer 5

Computer, with IP address and typically host (DNS) name

Server **application**, with **port** number, typically long running

Client **application**, often with a user interface, often transient

Application communication – **streams** (TCP) and/or **messages** (UDP)



# TCP and UDP: Streams and Messages

- Applications don't use IP directly
  - They use one or more transport protocols, usually TCP or UDP
- **TCP** provides a **reliable**, **bidirectional connection-oriented** service for **streams** of bytes
  - It uses failure masking techniques covered in a later lecture
- **UDP** provides a **best effort connectionless** service for **messages** = packets of bytes
  - Normally one-to-one but can do multicast; it uses IP directly
- Each application within a host is identified by a protocol-specific **port** number
  - Servers typically use well-known ports (e.g. associated with specific protocols or services, such as port 80 for HTTP)
  - Clients typically use random unused ports

# Message destinations

- If a client identifies a server using a specific IP address then the server must always have that IP address
  - E.g. the server process cannot be moved to another machine and the machine must always be on the same network
- If clients use names then a **name service** is used by the client to translate this to network location at run time
  - E.g. DNS for domain name to IP address; Broker service for service name to IP address/port number/etc. (see other lectures)

Q: Who has used UDP before?

# Java UDP Programming

# Address resolution

- Every OS also provides an API for host name resolution
- E.g. in Java, java.net.InetAddress

```
InetAddress aComputer =
  InetAddress.getByName("bruno.dcs.qmul.ac.uk");
```

- Internally the host will use DNS and/or other name services to look up the corresponding IP address
  - (See later lecture on DNS)

# Sockets

- A **socket** provides an abstract representation of a communication endpoint within a process
  - Required for TCP, UDP and also other protocols & mechanisms
  - Each protocol has its own specific type of socket
- The Java Sockets API just wraps the underlying OS sockets interfaces (as seen in C networking), so…
- A single **process** can have many **sockets**, but only one TCP server or UDP **socket** on a host can be bound to a given (IP and) **port** at any time
  - (except for multicast datagram sockets – see later lecture)
  - (Note, TCP client/connection sockets must match both the local AND remote ports and addresses)

# Java UDP API
(in java.net)

- **`DatagramPacket`** representing a UDP datagram with
  - Message bytes (array), length, remote address, remote port
- **`DatagramSocket`** representing a UDP socket
  - Bound to a specific port (or a random port)
  - **`send`**() and **`receive`**() methods for DatagramPackets
  - Configurable receive timeout
  - `connect`() optionally associate with a single remote address

You don't need to memorise the API, but you should understand the basic patterns of use

# UDP client: sends a message to the server and gets a reply

1. Make a **DatagramSocket** (random port)
2. Make a **DatagramPacket** with payload and destination (server) host & port
3. **.send** the DatagramPacket as a UDP packet
4. Make an empty **DatagramPacket**
5. **.receive** the next UDP packet into that DatagramPacket
6. **…**

```
import java.net.*;
import java.io.*;
public class UDPClient{
   public static void main(String args[]){
        // args give message contents and server hostname
        DatagramSocket aSocket = null;
         try {
                aSocket = new DatagramSocket();
                byte [] m = args[0].getBytes();
                InetAddress aHost = InetAddress.getByName(args[1]);
                int serverPort = 6789;
                DatagramPacket request = new DatagramPacket(m,  m.length(), aHost,
                                serverPort);
                aSocket.send(request);
                byte[] buffer = new byte[1000];
                DatagramPacket reply = new DatagramPacket(buffer, buffer.length);

                aSocket.receive(reply);
                System.out.println("Reply: " + new String(reply.getData()));
         } catch (SocketException e){System.out.println("Socket: " + e.getMessage());
         } catch (IOException e){System.out.println("IO: " + e.getMessage());
        } finally {if(aSocket != null) aSocket.close();}
   }
}
```

# UDP server: repeatedly receives a request and sends it back to the client (note, single-threaded)

1. Make a **DatagramSocket with known port**
2. (repeatedly…)
3. Make an empty **DatagramPacket**
4. **.receive** the next UDP packet into that DatagramPacket
5. **…** (handle request)
6. Make a **DatagramPacket** with payload and destination (client) host & port
7. **.send** the DatagramPacket as a UDP packet

```
import java.net.*;
import java.io.*;
public class UDPServer{
        public static void main(String args[]){
        DatagramSocket aSocket = null;
          try{
                aSocket = new DatagramSocket(6789);
                byte[] buffer = new byte[1000];
                while(true){
                  DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                  aSocket.receive(request);
                  DatagramPacket reply = new DatagramPacket(request.getData(),
                          request.getLength(), request.getAddress(), request.getPort());
                  aSocket.send(reply);
                }
           } catch (SocketException e){System.out.println("Socket: " + e.getMessage());
           } catch (IOException e) {System.out.println("IO: " + e.getMessage());}
        } finally {if(aSocket != null) aSocket.close();}
    }
}
```

# UDP considerations

- **Message size** – hard limit of 65508 bytes, but often 8k or less
  - Larger messages are broken up into smaller **fragments** on most physical networks (e.g. 1500 byte frames on Ethernet)
  - Reassembled by the receiver if/when ALL are received
  - => less reliable
- **Blocking** – receive normally blocks until a message is received (send normally returns immediately)
  - A socket **timeout** can be specified, e.g. to infer that a message might have been lost
- **Receive from any** – normally a socket will accept messages from any sender

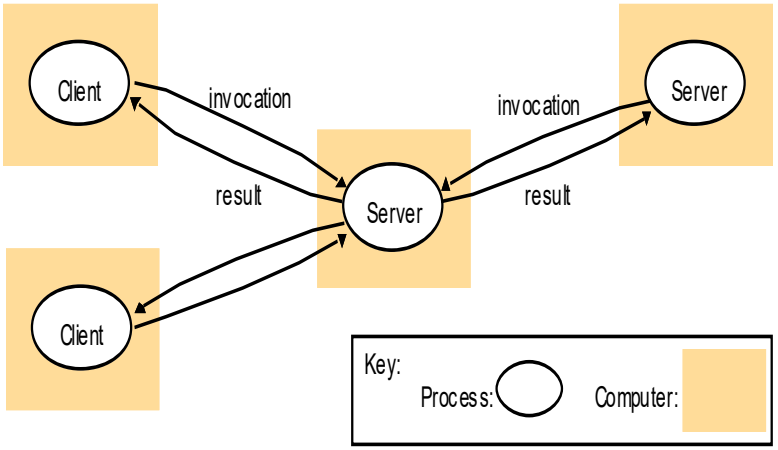
# *UDP considerations (continued)*

- **Failure Model** – **best effort** like IP
  - Checksums detect most **corruption** of messages and discard them
  - Messages may be **dropped**, e.g. due to corruption, no space in buffer at end system or in network (congestion)
  - Messages may arrive **out of order**
  - Messages may occasionally be duplicated
- **Uses** of UDP
  - OK for **small messages** and very **simple interactions**, e.g. DNS
    - Avoids **overheads** of TCP, e.g. avoiding connection establishment, avoiding recovery of all packets, avoiding in-order delivery of all information
    - More complex applications require *application* to implement flow and congestion control over UDP
  - Also supports **Multicast** & **broadcast** communication – see later lecture

Q: Who has defined an application protocol?

# Request-Reply Protocols

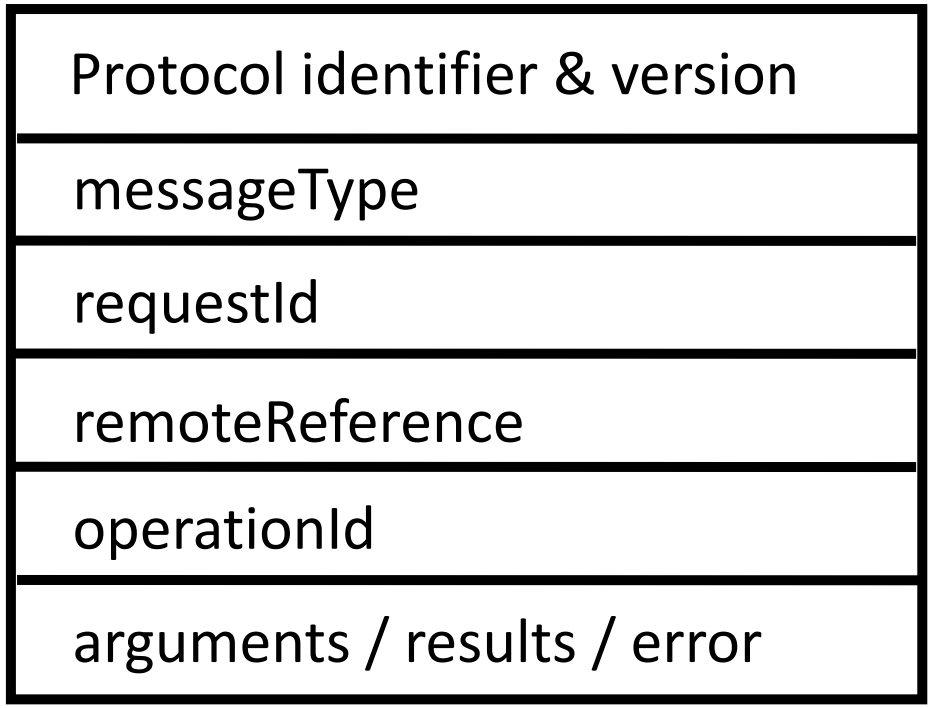
# Request-reply protocols (overview)




- Typical pattern for **client-server** interaction
  - Client sends request
  - Server receives request
  - Server handles request
  - Server sends response
  - Client receives response and continues
- Request and response are messages carried by selected interprocess communication mechanism (e.g. TCP, UDP)

# Example Request-reply message structure

Request/response all need to be encoded into **message**, for example (and this is just an example):




*String "MyProtocol:1"*

*int (0=Request, 1= Reply)*

*int*

*RemoteRef*

*int or Operation*

*array of bytes*

For example! Could be encoded as any unique byte sequence

May be inferred from process or socket receiving message

May not be required with TCP: if one request at a time or requests in order

May not be required if single interface per process/port

May not be required if single operation

May not be required if operation has no arguments. What encoding?

Types are just examples

# Request-reply message content notes

- **Protocol identifier** (& version) allows the server to check that it is using the same protocol as the client
- **Message type** (if needed) distinguishes requests & responses
- **Request ID** allows the client to match responses to requests and allows the server to identify duplicate requests
- **Remote Reference** (if required) distinguishes between different services within the same server process (using the same port)
- **Operation ID** identifies specific server operation to do
- Client and server must agree on how **arguments, results** and **errors** are encoded – see External Data Representation

# Request-reply Implementation notes

- Client normally **blocks** until the response is received
  - But alternatively protocols can be **reply-less** (one-way) or **asynchronous** (the reply is read by the client in a later operation) or **event-based**
- Can suffer from communication omission **faults** (i.e. lost request or reply) and process omission faults (i.e. server or client crash)
  - Different protocols may (or may not) try to **recover** from faults, e.g. with acknowledgements and retransmission (like TCP)

# Summary

# Distributed Systems (cont.) Summary

- Resources are typically encapsulated and selectively exposed over the network as **services** that are accessed by **clients**
- Clients and servers are typically both OS processes, and communicate directly with each other
- Common challenges for distributed systems:
  - **Heterogeneity** – coping with system component variability
  - **Failure Handling** – coping with partial failure
  - **Concurrency** – correctness and performance with concurrency

  (more challenges in a later lecture)

# Internet Summary

- The **Internet** is a global network of networks, based on the TCP/IP suite of open protocols
- Every **host** on the Internet has at least one **Internet** (**IP**) **address**
  - E.g. “128.243.22.73”
- Any host can send a **packet** or **datagram** of data to any other host with **best effort** service
  - i.e. packets may be lost, delayed, duplicated or corrupted (although normally these will be discarded)
- **Applications** (processes) within a single host are identified by protocol-specific **port** numbers
- Applications can communicate using
  - **TCP**, a **reliable connection-oriented bidirectional** byte **stream** service
  - **UDP**, a **best effort connectionless message** (datagram) service, which also supports **one-to-many** communication

# Java UDP Programming Summary

- A **Socket** represents a TCP or UDP communication end-point within a **process**
  - Associated with a particular protocol and port number
  - Optionally associated with a particular IP address
    - (otherwise will match any IP address [on the host])
- Java name resolution via java.net.**InetAddress**
- Java UDP API comprises java.net.**DatagramSocket** and java.net.**DatagramPacket**
- For low overhead applications with small messages and/or multicast (e.g. DNS, DHCP)

# Request-Reply Protocols Summary

- **Request-Reply** protocols are a natural fit for **client-server** interaction
- The requesting process must identify the **remote server** process or object (remote ref), **operation** to be performed, and provide any **arguments** (parameters)
- Request and reply messages typically include a **protocol identifier** to check compatibility, and a **request ID** to detect duplicate requests
- Client and server must agree on the **encoding** of messages and data (arguments, results, errors)

# Next steps

- Lecture 3 – Tuesday, 9am
  - Distributed Systems and Networking (part 2, inc. TCP)
- Lab 2 – next Thursday
  - Java TCP & UDP socket programming
- Look through the moodle resources,
  - Attempt past paper questions…
- Look at the text books, e.g. download the free one
  - https://www.distributed-systems.net/index.php/books/ds3/ds3-ebook/